

Autonomous Line-Following and Obstacle Avoiding Robot

Nitin Reddy Mereddy (202405815)

Sahil Sajan (202405476), Mahin Vishnu Kanur Balakrishnan (202403472),

Pradeep Reddy Lopinti (202407977), Saranya Kakarla (202408122),

Department of Applied Computer Science

CSCI 526: Embedded Systems

St. Francis Xavier University, Nova Scotia, Canada

I. INTRODUCTION

This practical project presents an autonomous mobile robot that follows a black line, avoids obstacles, and safely stops when conditions are unsafe. Built around an Arduino Uno (ATmega328P) and simulated in SimulIDE, the robot uses three IR line sensors, an HC-SR04 ultrasonic sensor, two DC motors driven through a ULN2001 array, and LEDs to indicate its operating mode. Its behaviour is structured as a Finite State Machine (FSM) with three modes: FOLLOW for proportional line tracking, AVOID for a predefined obstacle-bypass manoeuvre, and STOP for safe halting when the line is lost or avoidance fails. A cooperative scheduling scheme based on the Arduino millis() timer replaces blocking delays, with periodic tasks for sensor reading, FSM updates, motor control, and debug output to keep the system responsive and predictable. The report describes the design, circuit implementation, control logic, scheduling approach, and tests used to verify line following, obstacle avoidance, and stop behaviour.

II. PROJECT DESIGN CONCEPT

A. Objectives and Functional Requirements

The objective of this practical project is to design and implement an autonomous mobile robot that can follow a line, avoid obstacles, and stop safely in a simplified track environment. The robot follows a black line on a white surface using three IR sensors, detects obstacles ahead with an ultrasonic sensor, performs a basic avoidance manoeuvre, and enters a STOP state if the line is lost or the obstacle cannot be bypassed.

Its behaviour is organised as a Finite State Machine (FSM) with three modes: FOLLOW, where the robot tracks the line using proportional control based on the sensor readings; AVOID, where it temporarily leaves the line and executes a timed manoeuvre around an obstacle; and STOP, a safety mode entered when the line is missing for too long or avoidance fails. Across all modes, the design emphasises real-time behaviour, concurrent task execution, and reliable safe stopping without long blocking delays.

B. High-Level Behaviour

At a high level, the robot operates as a closed-loop control system that combines sensor feedback, FSM-based decision making, and motor actuation. During normal operation, the three IR line sensors determine whether the robot is centred on the line or drifting to the left or right. A simple proportional control law

$$\text{correction} = K_p \times \text{lineError}$$

adjusts the PWM signals sent to the left and right motors so that the robot smoothly steers back toward the centre of the track. As long as the line is visible and no obstacle is detected within the threshold distance, the FSM remains in the FOLLOW state and the robot continues along the path.

In parallel, the ultrasonic sensor periodically measures the distance to any object ahead of the robot. When this distance falls below the obstacle threshold, the FSM transitions from FOLLOW to AVOID. In AVOID mode, the robot executes a predefined manoeuvre: it briefly stops, turns, moves forward, and then turns back toward the line, continuing straight until the line sensors detect the track again. If the obstacle is cleared and the line is visible, the FSM returns to FOLLOW. If the line is not recovered or the manoeuvre takes too long, the FSM transitions to STOP to prevent unsafe behaviour.

If all three line sensors report white for longer than a defined timeout, the system assumes that the line has been lost, and the FSM enters the STOP state. In this mode, the motors are disabled and the STOP LED is turned on. The robot only leaves STOP when the line is visible again and there is no obstacle within the critical distance.

The overall behaviour relies on three core ideas: an FSM to structure the operating modes and transitions, proportional control for line following, and a cooperative scheduling scheme based on the Arduino millis() timer to run multiple tasks concurrently (sensor sampling, FSM updates, motor control, and debugging) without blocking delays. This organisation allows the system to remain responsive and predictable in real time.

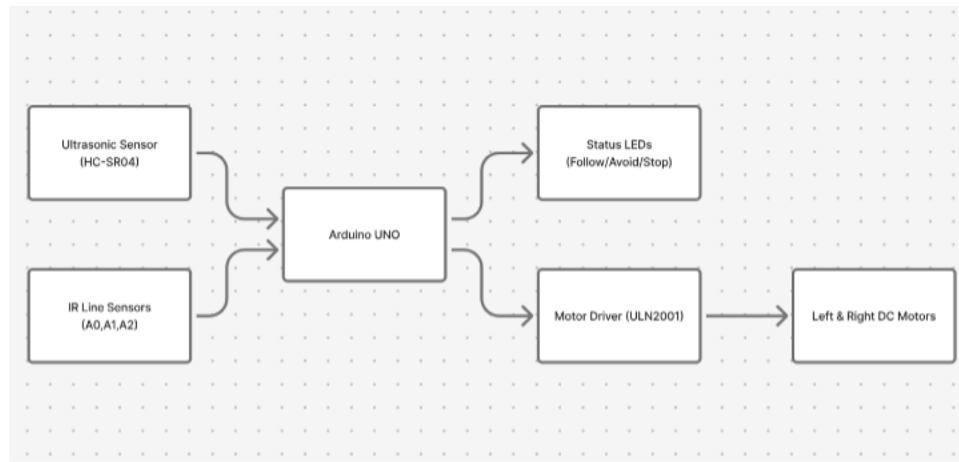


Fig. 1. System block diagram of the line-following and obstacle-avoiding robot, showing the main sensor, controller, and actuator subsystems.

III. CIRCUIT DESIGN AND IMPLEMENTATION

A. Components Overview

The hardware for this project is built around a simple but complete embedded control setup. All components are available inside SimulIDE and are chosen to match a realistic Arduino-based robot.

- **Microcontroller: Arduino Uno (ATmega328P)** The Arduino Uno acts as the main controller. It reads all sensor inputs, runs the Finite State Machine (FSM) logic, computes motor commands for line following and obstacle avoidance, and drives the status LEDs.
- **Line Sensors: 3 × IR-simulated switches on A0, A1, A2** Three simulated IR line sensors are used to detect a black line on a white background. In SimulIDE, each sensor is implemented as a switch connected to 5 V, with a 10 kΩ pull-down resistor to ground.
 - Left sensor → analog pin A0
 - Centre sensor → analog pin A1
 - Right sensor → analog pin A2

When a sensor is over the black line, the switch is turned ON (logic HIGH). When it is over the white surface, the switch is OFF (logic LOW).

- **Ultrasonic Sensor: HC-SR04** An HC-SR04 ultrasonic sensor is used to measure the distance to any obstacle in front of the robot.
 - TRIG pin → digital pin D8
 - ECHO pin → digital pin D9
 - VCC → 5 V, GND → GND

In SimulIDE, the obstacle distance can be simulated using the *In v=m* pin connected to a potentiometer.

- **Motor Driver: ULN2001 + Two DC Motors** Two DC motors are used to drive the left and right wheels. They are not connected directly to the Arduino pins. Instead, the PWM outputs from the Arduino go into a ULN2001 transistor array, which acts as a low-side driver and provides enough current for the motors.
 - Arduino D5 → IN1 of ULN2001 (left motor)
 - Arduino D6 → IN2 of ULN2001 (right motor)
 - OUT1 of ULN2001 → negative terminal of left motor
 - OUT2 of ULN2001 → negative terminal of right motor
 - Motor positive terminals → motor battery +

The ULN2001 also includes built-in flyback diodes, which protect the circuit from voltage spikes when the motors are switched on and off.

- **Mode Indicator LEDs** Three LEDs are used to show the current operating mode of the robot:
 - FOLLOW mode LED → digital pin D10
 - AVOID mode LED → digital pin D11
 - STOP mode LED → digital pin D12

Each LED is connected through a $220\ \Omega$ resistor to ground. This makes it easy to see, at a glance, whether the robot is following the line, avoiding an obstacle, or stopped.

- **Power Supply and Common Ground** In the simulation, the motors are powered by a separate 6 V battery, while the Arduino is powered from its own 5 V source. The motor battery positive is connected to the motor positive terminals and to the COM pin of the ULN2001. All grounds (Arduino GND, ULN2001 GND, motor battery negative, sensor grounds, and LED grounds) are tied together to form a single common reference.

Figure 2 shows the complete circuit as implemented in the SimulIDE workspace, including the Arduino, sensors, motor driver, motors, and indicator LEDs.

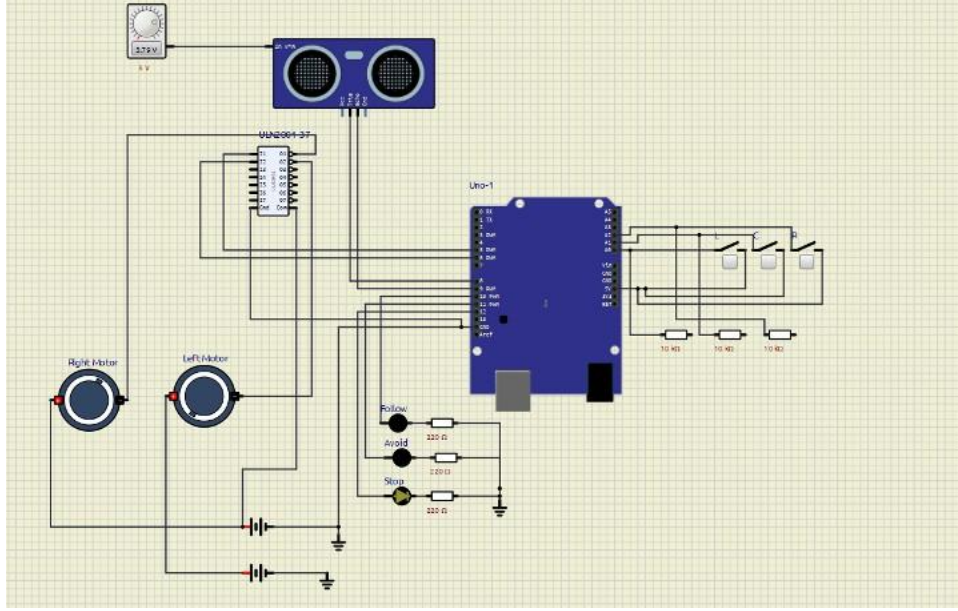


Fig. 2. SimulIDE circuit layout for the line-following and obstacle-avoiding robot, showing the Arduino Uno, line sensors, ultrasonic sensor, ULN2001 motor driver, DC motors, and mode indicator LEDs.

B. Circuit Connections

This subsection describes how each group of components is wired to the Arduino and to each other. The goal is to make the wiring easy to understand and to highlight the connections that are most important for correct operation.

1) **Line Sensor Interface:** Each line sensor is modelled as a switch to 5 V with a pull-down resistor. When the robot is on the line, the switch closes and the corresponding Arduino pin reads HIGH; otherwise it reads LOW. The connections are summarised in Table I.

TABLE I LINE SENSOR CONNECTIONS TO ARDUINO		
Sensor	Arduino Pin	Additional Connections
Left	A0	10 k Ω resistor to GND, switch to 5 V
Centre	A1	10 k Ω resistor to GND, switch to 5 V
Right	A2	10 k Ω resistor to GND, switch to 5 V

This simple arrangement ensures a clean digital reading: either 0 V (white surface) or 5 V (black line). The software then interprets these patterns to compute the line error value used by the proportional controller.

2) **Ultrasonic Sensor Interface:** The HC-SR04 ultrasonic sensor uses one pin to trigger a pulse and another to measure the echo time. The wiring is:

- TRIG $\rightarrow$ D8 (output from Arduino)
- ECHO $\rightarrow$ D9 (input to Arduino)
- VCC $\rightarrow$ 5 V
- GND $\rightarrow$ common ground

In the sketch, the TRIG pin is briefly pulsed HIGH, and the ECHO pulse width is measured using pulseIn(). This duration is converted into a distance in centimetres, which is stored in the obstacleDistanceCm variable. In SimulIDE, the sensed distance is controlled by the *In v=m* pin of the ultrasonic module, typically driven by a potentiometer so that different obstacle distances can be tested easily.

3) *Motor Driver and Motors*: The Arduino cannot drive the motors directly, so PWM signals on pins D5 and D6 are passed through the ULN2001 transistor array, which switches the motor currents on the low side. The key connections are:

- Arduino D5 → IN1 (ULN2001 pin 1) → OUT1 (pin 16) → left motor negative
- Arduino D6 → IN2 (ULN2001 pin 2) → OUT2 (pin 15) → right motor negative
- Left and right motor positive terminals → motor battery positive
- ULN2001 GND (pin 8) → common ground
- ULN2001 COM (pin 9) → motor battery positive

Connecting the COM pin to the motor battery positive is important, because it enables the internal flyback diodes in the ULN2001. These diodes protect the circuit from voltage spikes generated when the motors are switched off. In this configuration, the Arduino controls motor speed and direction (forward or off) by varying the PWM duty cycle on D5 and D6, while the ULN2001 safely handles the higher currents.

4) *Mode Indicator LEDs*: The three mode LEDs are each connected to a dedicated digital output on the Arduino Uno. The wiring is shown in Table II.

TABLE II
MODE INDICATOR LED CONNECTIONS

Function	Arduino Pin	Connection
FOLLOW mode LED	D10	LED anode to D10, cathode through 220Ω resistor to GND
AVOID mode LED	D11	LED anode to D11, cathode through 220Ω resistor to GND
STOP mode LED	D12	LED anode to D12, cathode through 220Ω resistor to GND

The software turns these LEDs on and off inside the `updateStateLEDs()` function, so that the current FSM state is always visible while testing.

- 5) *Power Supply and Common Ground*: Finally, all parts of the circuit share a common reference point. The main rules are:
- Arduino GND, motor battery negative, ULN2001 GND, sensor grounds, and LED grounds are all connected together.
 - The motor battery positive is connected to the motor positive terminals and to the COM pin of the ULN2001.
 - The Arduino 5 V supply powers the logic side: the line sensors, ultrasonic sensor, and LED anodes.

Using a common ground is essential. It ensures that when the Arduino outputs a HIGH signal, the other components interpret it correctly, and it keeps the sensor readings stable. This wiring arrangement in SimulIDE closely matches how the circuit would be wired in a real hardware implementation.

IV. FINITE STATE MACHINE (FSM) AND CONTROL LOGIC

The overall behaviour of the robot is organised using a Finite State Machine (FSM). The FSM groups the robot's operation into a small number of clearly defined modes and specifies when the system should switch from one mode to another. This makes the control logic easier to design, test, and reason about, especially when several different conditions (line status, obstacle distance, and timeouts) must be considered at the same time.

A. State Definitions

The robot uses three main states: FOLLOW, AVOID, and STOP. Each state represents a distinct mode of operation with its own control logic and clear entry and exit conditions.

FOLLOW State: The FOLLOW state is the normal operating mode of the robot. In this state, the robot follows the black line using the three IR sensors and a proportional controller. The software reads the line sensors, computes a discrete line error value (`lineError`), and adjusts the left and right motor speeds so that the robot stays centred on the track. While the robot is in FOLLOW, two main conditions are monitored:

- Obstacle detection: If the measured obstacle distance (`obstacleDistanceCm`) falls below a predefined threshold (`OBSTACLE_THRESHOLD_CM`), the FSM transitions from FOLLOW to AVOID. This means the robot has detected something in front of it and must perform an avoidance manoeuvre.
- Line loss: If all three line sensors read LOW at the same time, the variable `lineLost` becomes true. A timer is started and, if the line remains lost for longer than `LINE_LOST_TIMEOUT` milliseconds, the FSM transitions from FOLLOW to STOP to prevent the robot from driving blindly.

AVOID State: The AVOID state is entered when an obstacle is detected in front of the robot while it is following the line. In this state, the robot temporarily suspends line following and performs a fixed, step-by-step avoidance manoeuvre. The manoeuvre consists of stopping briefly, turning, and moving forward in a pattern that causes the robot to move around the obstacle and then attempt to return to the line.

While in AVOID, the system monitors both time and sensor feedback:

- Successful avoidance: If the obstacle distance becomes greater than or equal to `OBSTACLE_THRESHOLD_CM` and the line is visible again (`lineLost` is false), and a minimum amount of time in `AVOID` has passed, the FSM transitions back to `FOLLOW`. This indicates that the obstacle has been cleared and the robot has rejoined the track.
- Avoidance failure or timeout: If the total time spent in `AVOID` exceeds `AVOID_TOTAL_TIMEOUT`, the FSM transitions to `STOP`. This prevents the robot from remaining in an unsuccessful avoidance loop.

STOP State: The `STOP` state is a safety mode. The robot enters this state if the line is lost for too long in `FOLLOW`, or if the obstacle avoidance process takes too long in `AVOID`. In `STOP`, the motors are turned off, and the `STOP` indicator LED is turned on so that the state is clearly visible in the simulation.

The system remains in `STOP` until both of the following conditions are met:

- The line is visible again (at least one line sensor is `HIGH`, so `lineLost` is false).
- There is no obstacle within the critical distance (the measured `obstacleDistanceCm` is greater than or equal to `OBSTACLE_THRESHOLD_CM`).

When these conditions are satisfied, the FSM transitions from `STOP` back to `FOLLOW`, and the robot resumes normal line following.

B. State Transitions and Conditions

The state transitions are implemented in the `updateFSM()` function, which is called periodically based on the scheduler. The function uses a switch statement on the current state and checks a small number of key variables:

- `obstacleDistanceCm` and `OBSTACLE_THRESHOLD_CM`: determine whether an obstacle is close enough to trigger `AVOID` or prevent a return to `FOLLOW`.
- `lineLost` and `LINE_LOST_TIMEOUT`: detect when the line has been missing for too long, triggering a transition to `STOP`.
- `AVOID_TOTAL_TIMEOUT`: sets an upper limit on how long the robot is allowed to stay in the `AVOID` state.

The simplified transition logic can be described as follows:

- From `FOLLOW`:
 - If `obstacleDistanceCm < OBSTACLE_THRESHOLD_CM` → state changes to `AVOID`.
 - If `lineLost` remains true for more than `LINE_LOST_TIMEOUT` ms → state changes to `STOP`.
- From `AVOID`:
 - If total time in `AVOID` exceeds `AVOID_TOTAL_TIMEOUT` → state changes to `STOP`.
 - If `lineLost` is false and `obstacleDistanceCm ≥ OBSTACLE_THRESHOLD_CM` and a minimum time has passed in `AVOID` → state changes back to `FOLLOW`.
- From `STOP`:
 - If `lineLost` is false and `obstacleDistanceCm ≥ OBSTACLE_THRESHOLD_CM` → state changes to `FOLLOW`.

Each time the state changes, the function `updateStateLEDs()` is called to update the three mode LEDs so that the current state is always visible during testing.

The three main operating modes and their transitions are summarised in the FSM diagram in Figure 3.

C. Control Logic for Line Following and Avoidance

The control logic defines how the robot drives its motors inside each state. The two most important cases are the line following behaviour in the `FOLLOW` state and the avoidance manoeuvre in the `AVOID` state.

Line Following Control: The line following behaviour uses the three IR sensors to estimate how far the robot is from the centre of the line. The digital readings from the left, centre, and right sensors are combined into a discrete `lineError` value:

- `lineError = -1` when the line is mainly under the left sensor (robot is too far to the right).
- `lineError = 0` when the line is under the centre sensor (robot is centred).
- `lineError = +1` when the line is mainly under the right sensor (robot is too far to the left).

Intermediate patterns (for example, two sensors active at once) are mapped to the closest of these three values. If all three sensors read `LOW`, the line is considered lost, and `lineLost` is set to true.

When the line is visible, the motors are controlled by a simple proportional controller:

$$\text{correction} = K_p \times \text{lineError}$$

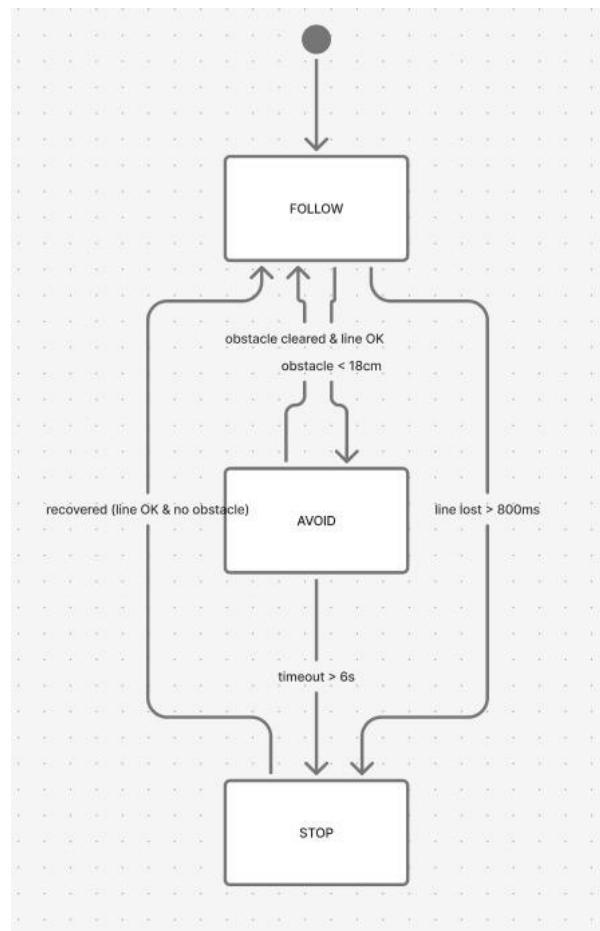


Fig. 3. Finite State Machine for FOLLOW, AVOID, and STOP modes, showing the main transitions based on obstacle distance, line loss, and timeout conditions.

$$\text{leftSpeed} = \text{BASE_SPEED} - \text{correction}, \quad \text{rightSpeed} = \text{BASE_SPEED} + \text{correction}.$$

Here, BASE_SPEED is the nominal PWM value for both motors, and K_p is a proportional gain. If the robot drifts to the right (lineError = -1), the left motor slows down and the right motor speeds up, turning the robot back toward the line. If it drifts to the left, the opposite happens. When lineError = 0, both motors run at the same speed, and the robot moves straight ahead.

The computed speeds are clamped to the valid PWM range (0 to 255) and then applied using analogWrite() in the setMotors() function. If the line is lost, the controller stops the motors temporarily while the FSM decides whether to remain in FOLLOW or transition to STOP.

Obstacle Avoidance Sequence: In the AVOID state, normal line following is suspended, and the robot carries out a fixed avoidance sequence. The sequence is implemented using the integer variable avoidStep and the timestamp avoidStepStart:

- Step 0: Stop briefly Both motors are set to speed 0. The robot pauses for a short time (for example, 200 ms) to stabilise before turning.
- Step 1: Turn right The left motor is turned ON at BASE_SPEED, while the right motor is turned OFF. This causes the robot to pivot to the right. After a fixed duration (e.g., 700 ms), the code advances to the next step.
- Step 2: Move forward Both motors are turned ON at BASE_SPEED, causing the robot to move straight ahead past the obstacle. After another timed interval (e.g., 900 ms), the sequence continues.
- Step 3: Turn left The right motor is turned ON and the left motor is turned OFF, making the robot pivot back to the left. This aims to bring the robot back toward the line on the other side of the obstacle.
- Step 4: Move forward until recovery Both motors are again set to BASE_SPEED. The robot continues straight until the FSM conditions indicate that the obstacle has been cleared and the line has been found again, at which point the state changes back to FOLLOW.

Each step uses the current time (from millis()) and avoidStepStart to decide when to move on to the next step. This avoids blocking delays and keeps the behaviour compatible with the cooperative scheduling approach. If, for any reason, the avoidance

sequence takes too long (longer than `AVOID_TOTAL_TIMEOUT`), the FSM switches to STOP to avoid unsafe or undefined behaviour.

By combining the FSM structure with proportional control in FOLLOW and a timed manoeuvre in AVOID, the robot achieves clear, predictable behaviour in all three operating modes.

V. CONCURRENCY, SCHEDULING, AND RESPONSIVENESS

A. Cooperative Scheduling with `millis()`

To meet the real-time requirements of line following and obstacle avoidance, the robot uses a simple cooperative scheduling approach based on the Arduino `millis()` timer. Instead of using blocking `delay()` calls, the `loop()` function behaves like a scheduler that periodically checks whether it is time to run each task.

For every task in the system (line sensor reading, ultrasonic distance measurement, FSM update, motor control update, and debug printing), a separate timestamp is stored. On each pass through `loop()`, the current time from `millis()` is compared with the last execution time of each task. If enough time has passed, that task is executed and its timestamp is updated. This design allows multiple activities to run “in parallel” from the point of view of the robot, without any single function blocking the others.

This cooperative scheduling method has two main advantages. First, it keeps the system responsive, because sensor inputs and control outputs are refreshed at regular, short intervals. Second, it makes the timing behaviour explicit and easy to adjust by changing the task periods, rather than scattering fixed delays throughout the code.

B. Task Periods and Timing Choices

The main periodic tasks and their execution periods are summarised in Table III.

TABLE III
PERIODIC TASKS AND THEIR EXECUTION PERIODS

Task	Period (ms)
Read line sensors	10
Read ultrasonic sensor	60
Update FSM	10
Motor control update	10
Debug serial prints	500

The line sensors, FSM update, and motor control update all run every 10 ms. This relatively fast rate is chosen so that the robot can quickly detect changes in the line position and adjust its motor speeds, resulting in smooth line following. The ultrasonic sensor is read less frequently (every 60 ms) because obstacle distances do not need to be updated as rapidly and the sensor is inherently slower. Debug printing runs only every 500 ms to avoid flooding the serial output and to minimise its impact on the main control loop.

These timing choices strike a balance between responsiveness and computational load. The most time-critical tasks are updated frequently, while slower or non-critical tasks are scheduled more sparsely.

C. Responsiveness Improvements

Several design decisions were made specifically to improve responsiveness and robustness:

- **No blocking `delay()`:** By avoiding blocking delays in the main behaviour, the robot can continue to read sensors and update the FSM even while performing obstacle avoidance or waiting for conditions to change. This leads to faster reaction times and more predictable timing.
- **Stable distance measurements:** Although the ultrasonic readings are relatively simple, they are taken at a fixed period and interpreted in a consistent way. This helps reduce sudden jumps or jitter in the perceived obstacle distance and prevents unnecessary state changes.
- **Robust line sensing:** The IR sensor readings are converted into a discrete line error and a separate `lineLost` flag. Treating the sensor patterns in this way effectively debounces the line detection logic and avoids rapid oscillations between “line seen” and “line lost” when the robot is near the edge of the track.
- **Avoidance timeout (`AVOID_TOTAL_TIMEOUT`):** A maximum time limit is placed on the AVOID state. If the robot is unable to complete its avoidance manoeuvre within this time, it transitions to STOP. This prevents the system from getting stuck indefinitely in an avoidance loop.
- **Line lost timeout (`LINE_LOST_TIMEOUT`):** When the line disappears, a timer is started. Only if the line remains lost longer than this timeout does the system enter STOP. This allows the robot to tolerate brief gaps or noise without overreacting, while still responding quickly when the line is genuinely lost.

Together, the cooperative scheduler, carefully chosen task periods, and explicit timeout mechanisms ensure that the robot remains responsive to sensor changes, avoids infinite loops, and behaves safely in abnormal situations.

VI. TESTING AND EVALUATION

This section describes how the robot was tested in SimulIDE and how its behaviour was evaluated in the three main scenarios: normal line following, obstacle detection and avoidance, and stop mode. Additional observations were made using the serial console output.

A. Test Setup

All tests were performed in the SimulIDE environment using the complete circuit described earlier. The three line sensors were simulated using switches connected to the Arduino analog pins A0–A2. By toggling these switches, different line positions (left, centre, right, or line lost) were reproduced.

The ultrasonic sensor distance was controlled via the $In\ v=m$ pin, driven by a potentiometer. Adjusting the potentiometer changed the apparent distance to an obstacle, allowing precise triggering of the obstacle threshold. During each test, the following outputs were observed:

- The three mode LEDs (FOLLOW, AVOID, STOP) on pins D10–D12.
- The robot's motor activity and motion in the SimulIDE workspace.
- Serial monitor messages indicating the current state, line error, lineLost flag, and measured distance.

B. Scenario 1 – Normal Line Following

Procedure: The centre line sensor was kept HIGH to simulate the robot being correctly positioned on the line. The left and right sensors were then toggled to simulate small deviations of the robot to each side of the track.

Expected Behaviour: The FSM should remain in the FOLLOW state. The motors should adjust their speeds smoothly to correct any left or right drift, and the FOLLOW LED should remain ON. The serial output should show STATE = FOLLOW with lineError values of -1 , 0 , or $+1$ depending on which sensors were active.

Observed Behaviour: The observed behaviour matched expectations. The robot stayed on the line, corrected its position when the side sensors were activated, and the FOLLOW LED remained lit. No unexpected state changes or oscillations were observed.

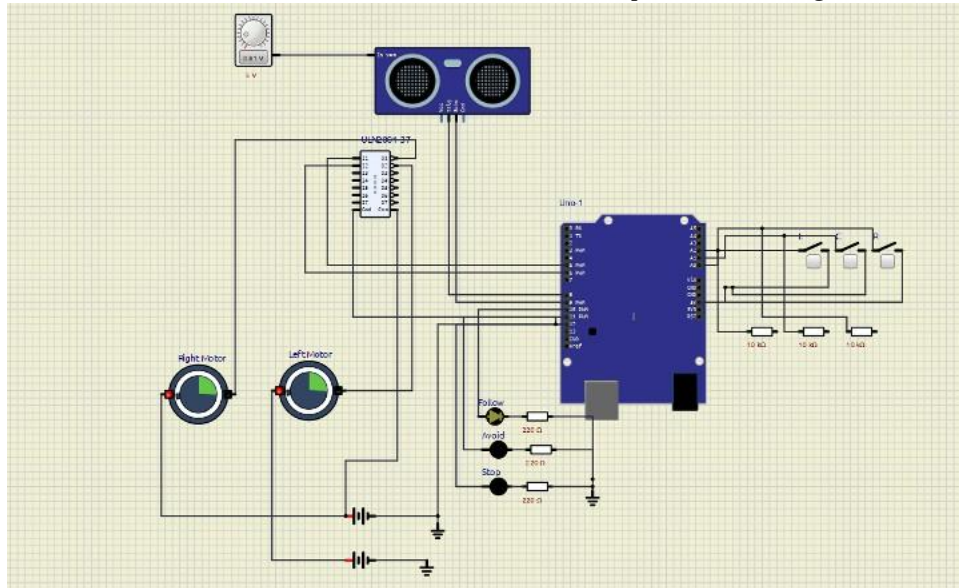


Fig. 4. Simulation screenshot showing the robot in FOLLOW mode while tracking the line.

C. Scenario 2 – Obstacle Detection and Avoidance

Procedure: With the robot following the line, the potentiometer connected to the ultrasonic sensor was adjusted so that the measured distance dropped below the obstacle threshold.

Expected Behaviour: The FSM should transition from FOLLOW to AVOID. The AVOID LED should turn ON, and the robot should execute the defined avoidance sequence (stop, turn right, move forward, turn left, then move forward) before attempting to rejoin the line. When the obstacle is cleared and the line is visible, the state should return to FOLLOW.

Observed Behaviour: The robot reliably switched to AVOID when the distance threshold was crossed. The avoidance manoeuvre completed as expected, and the robot returned to FOLLOW once the simulated obstacle was removed and the line sensors detected the track again. Minor variations in the exact path were observed, but the overall behaviour remained consistent.

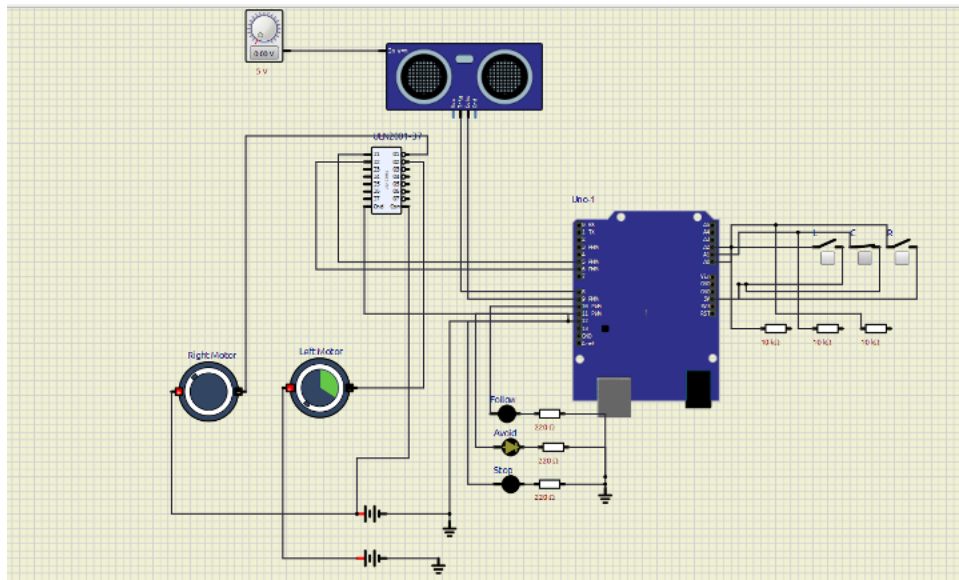


Fig. 5. Simulation screenshot showing the robot performing an obstacle avoidance manoeuvre in AVOID mode.

D. Scenario 3 – Stop Mode

Procedure: All three line sensor switches were turned OFF simultaneously for more than one second to simulate the robot losing the line completely.

Expected Behaviour: After the line has been lost for longer than the configured `LINE_LOST_TIMEOUT`, the FSM should transition to STOP. The STOP LED should turn ON, both motors should stop, and the serial output should show `STATE = STOP`. The robot should remain in STOP until the line is visible again and no obstacle is detected.

Observed Behaviour: The system behaved as expected. After the timeout, the motors were disabled, the STOP LED lit up, and the console displayed the STOP state. Restoring one of the line sensors and ensuring the obstacle distance was safe allowed the system to return automatically to FOLLOW.

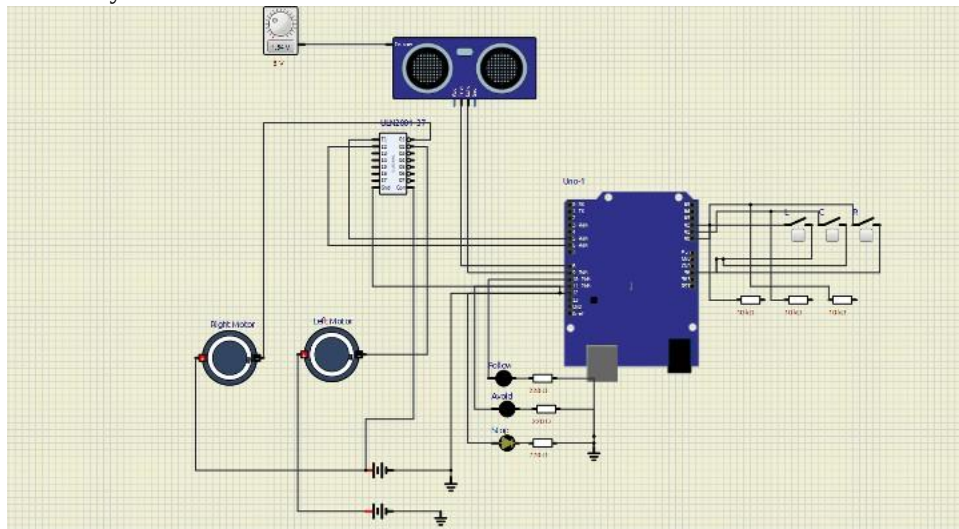
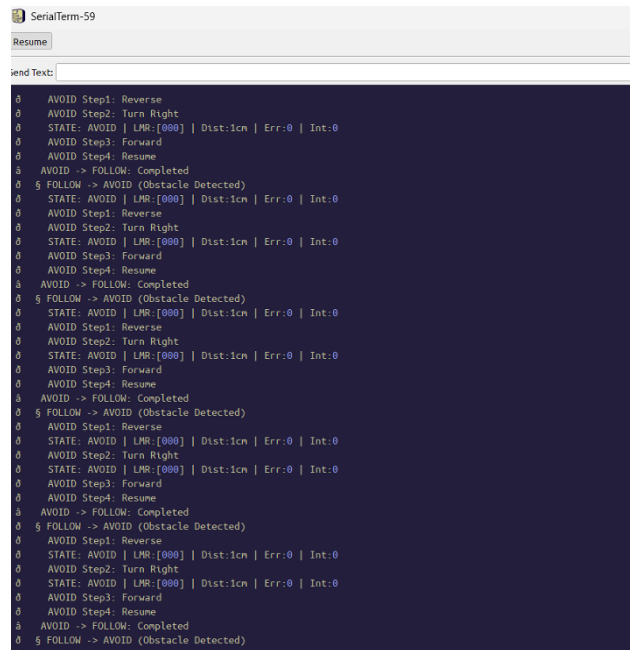


Fig. 6. Simulation screenshot showing the robot in STOP mode after the line has been lost.

E. Console and Debug Output

The serial console was used throughout testing to monitor internal variables in real time. Each debug line reported the current FSM state, the `lineError` value, whether the line was considered lost, the measured obstacle distance in centimetres, and the current left and right motor speeds. This made it straightforward to verify that state transitions occurred at the correct thresholds and that the control logic responded as designed.



```

SerialTerm-59
Resume

Send Text:

⌘ AVOID Step1: Reverse
⌘ AVOID Step2: Turn Right
⌘ STATE: AVOID | LMR:[880] | Dist:1cm | Err:0 | Int:0
⌘ AVOID Step3: Forward
⌘ AVOID Step4: Resume
⌘ AVOID -> FOLLOW: Completed
⌘ $ FOLLOW -> AVOID (Obstacle Detected)
⌘ STATE: AVOID | LMR:[880] | Dist:1cm | Err:0 | Int:0
⌘ AVOID Step1: Reverse
⌘ AVOID Step2: Turn Right
⌘ STATE: AVOID | LMR:[880] | Dist:1cm | Err:0 | Int:0
⌘ AVOID Step3: Forward
⌘ AVOID Step4: Resume
⌘ AVOID -> FOLLOW: Completed
⌘ $ FOLLOW -> AVOID (Obstacle Detected)
⌘ STATE: AVOID | LMR:[880] | Dist:1cm | Err:0 | Int:0
⌘ AVOID Step1: Reverse
⌘ AVOID Step2: Turn Right
⌘ STATE: AVOID | LMR:[880] | Dist:1cm | Err:0 | Int:0
⌘ AVOID Step3: Forward
⌘ AVOID Step4: Resume
⌘ AVOID -> FOLLOW: Completed
⌘ $ FOLLOW -> AVOID (Obstacle Detected)
⌘ STATE: AVOID | LMR:[880] | Dist:1cm | Err:0 | Int:0
⌘ AVOID Step1: Reverse
⌘ AVOID Step2: Turn Right
⌘ STATE: AVOID | LMR:[880] | Dist:1cm | Err:0 | Int:0
⌘ AVOID Step3: Forward
⌘ AVOID Step4: Resume
⌘ AVOID -> FOLLOW: Completed
⌘ $ FOLLOW -> AVOID (Obstacle Detected)

```

Fig. 7. Serial console output showing FSM state, line error, lineLost flag, obstacle distance, and motor speeds during testing.

F. Evaluation and Discussion

The tests showed that the system is generally responsive and stable: the cooperative scheduler allowed the robot to react quickly to changes in line position and obstacle distance, and state transitions occurred cleanly without oscillation between modes. Some limitations remain. Because the ULN2001 can only sink current, the robot cannot reverse, so obstacle avoidance is limited to forward and turning motions. The line control uses a simple proportional strategy rather than a full PID controller, and all thresholds were tuned for this specific simulated track. Even with these constraints, the implementation satisfies the practical requirements for line following, obstacle avoidance, and safe stopping in the chosen test environment.

VII. CONCLUSION

This practical project implemented and evaluated an autonomous line-following and obstacle-avoiding robot using an Arduino Uno and a SimulIDE-based circuit. The system successfully met the functional requirements: it was able to follow a black line using three IR sensors, detect obstacles with an ultrasonic sensor, perform a predefined avoidance manoeuvre when an obstacle was present, and enter a safe STOP mode when the line was lost or avoidance failed. Throughout operation, the mode indicator LEDs and serial debug messages provided clear feedback on the robot's internal state.

The use of a Finite State Machine (FSM) with three states (FOLLOW, AVOID, STOP) made the control logic structured and easy to reason about. Combined with proportional control for line following and a cooperative scheduling scheme based on `millis()`, the robot maintained responsive and predictable behaviour without relying on blocking delays. The periodic tasks for sensor reading, state updates, and motor control ensured that important events were handled in a timely manner, while the timeout mechanisms prevented the system from becoming stuck in unsafe or undefined conditions.

There are several natural directions for future work. The proportional line-following controller could be extended to a full PID controller, which would allow finer tuning of the robot's response on more complex tracks. Replacing the ULN2001 with an H-bridge motor driver would enable true bidirectional control, making it possible to add reverse motion and more flexible avoidance strategies. Finally, the test scenarios could be expanded to include more varied obstacle layouts, intersections, and curves, bringing the system closer to a real-world mobile robot application.

REFERENCES

- [1] C. Koutras, D. Dimitropoulos, A. Stavridis, and G. Tsihrintzis, "A Versatile Approach to Polygonal Object Avoidance in Indoor Environments with Hardware Schemes Using an FPGA-Based Multi-Robot," *Sensors*, vol. 23, no. 6, pp. 1–22, 2023.
- [2] R. S. García, D. Ordonez, and J. C. González, "Design and Implementation of an Asynchronous Finite State Controller for Wheeled Mobile Robots," *Actuators*, vol. 11, no. 11, pp. 1–20, 2022.
- [3] S. O. Oladimeji and B. B. Adewumi, "Design and Implementation of an Improved Obstacle Avoidance Model for Land Mower," *International Journal of Information Technology and Computer Science*, vol. 14, no. 5, pp. 1–10, 2022.
- [4] J. Ma, "Obstacle Detection and Avoidance Using Ultrasonic Sensors in Autonomous Robots," *Highlights in Science, Engineering and Technology*, vol. 71, pp. 68–74, 2023.
- [5] R. Farkh and K. Aljaloud, "Vision Navigation Based PID Control for Line Tracking Robot," *Intelligent Automation & Soft Computing*, vol. 35, no. 1, pp. 901–916, 2023.